MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Project

Software Design

Author: Ghile Patricia-Maria
Group: 30433

Faculty of Automation and Computer Science

May 2026

Contents

1 Problem definition **2**

2 Analysis phase - Use cases & requirements **2**
2.1 Functional requirements 2
2.2 Use cases 3

3 Design phase **3**
3.1 Architecture overview 3
3.2 Entity-relationship diagram 3
3.3 Client Application 4
3.4 Backend Services 4
3.5 Class diagrams 4
3.6 Sequence diagrams 4
3.7 Design patterns 6

4 Implementation phase - Application **6**
4.1 Tools and technologies 6
4.2 Application structure 7
4.3 User manual 7

5 Testing and validation **7**

1 Problem definition

The objective of this project is to implement a complete client-server application that extends the Eurovision Management System developed in previous assignments. The system transitions to a distributed architecture, introducing a clear separation between the client-side user interface and the backend services responsible for business logic, persistence, and authorization.

The domain remains focused on managing the Eurovision Song Contest catalog. This phase introduces new capabilities: standard request-response communication via REST for CRUD operations, real-time bidirectional communication using WebSockets for a live chat functionality, internationalization for a multi-language UI, and graceful error handling.

The following sections detail the development phases:

- **Analysis phase:** Defines the functional requirements, including client-server interactions, real-time chat, and internationalization.
- **Design phase:** Details the client-server architecture, database schemas, and system components.
- **Implementation phase:** Describes the technology stack (Javalin, WebSockets, HTML/CSS/JavaScript) and project structure.
- **Testing and validation:** Outlines the verification of API endpoints and real-time features.

Deliverables for this assignment include: the GitHub repository containing the source code (with separate client and server modules), independent PostgreSQL schemas, and this documentation featuring Use Case, ERD, Class, and Sequence diagrams.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

The system provides the following functional requirements:

- **Client-Server Architecture:** A clear separation between the client web application and the server's exposed APIs.
- **Core Functionality (Legacy):** Users can authenticate, perform CRUD operations, and export data in CSV, JSON, and XML formats.
- **Internationalization:** The system provides a multi-language user interface (English and Romanian) that can be switched dynamically.
- **Real-time Chat:** Users can send messages that are broadcasted to all connected clients instantly via WebSockets, displaying the sender and timestamp without refreshing the page.
- **Graceful Error Handling:** The server returns structured error objects (message, code), and the client captures and displays these meaningfully in the UI without crashing.

2.2 Use cases

The system involves Visitors, Administrators, and Heads of Delegation (HOD). The interactions between these actors and the system in order to accomplish specific tasks (such as managing entries, chatting live, or exporting data) are illustrated in the use case diagram below.

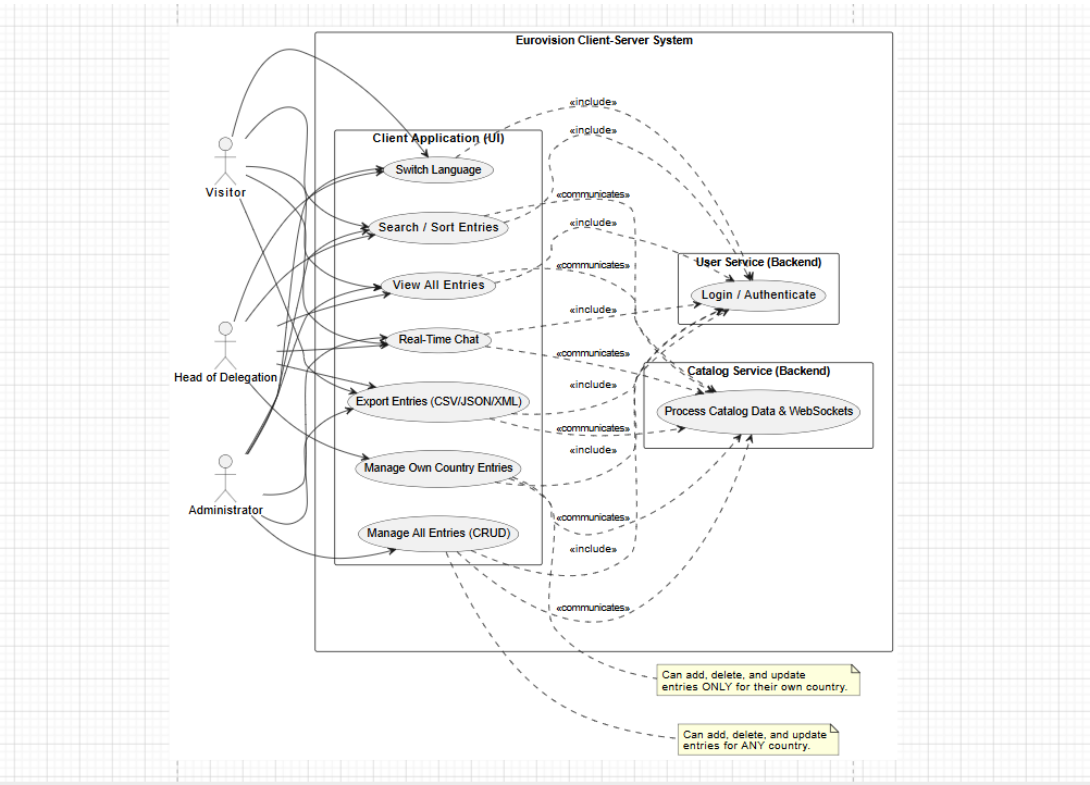


Figure 1: System Use Case Diagram

3 Design phase

3.1 Architecture overview

The system follows a Client-Server Architecture based on microservices. The frontend (client) is developed as a standalone web application using HTML, CSS, and JavaScript. It communicates with two independent backend services (`UserService` and `CatalogService`) via standard HTTP/REST endpoints for data retrieval/modification, and via WebSockets for real-time chat features.

This decoupling ensures that the client can be deployed independently from the backend servers and interacts with them strictly through the exposed APIs.

3.2 Entity-relationship diagram

To ensure service independence, the database is split into distinct schemas:

- **eurovision_users:** Stores authentication data, roles, and country affiliations (managed by `UserService`).

- **eurovision_catalog:** Stores Eurovision entries and contest metadata (managed by `CatalogService`).

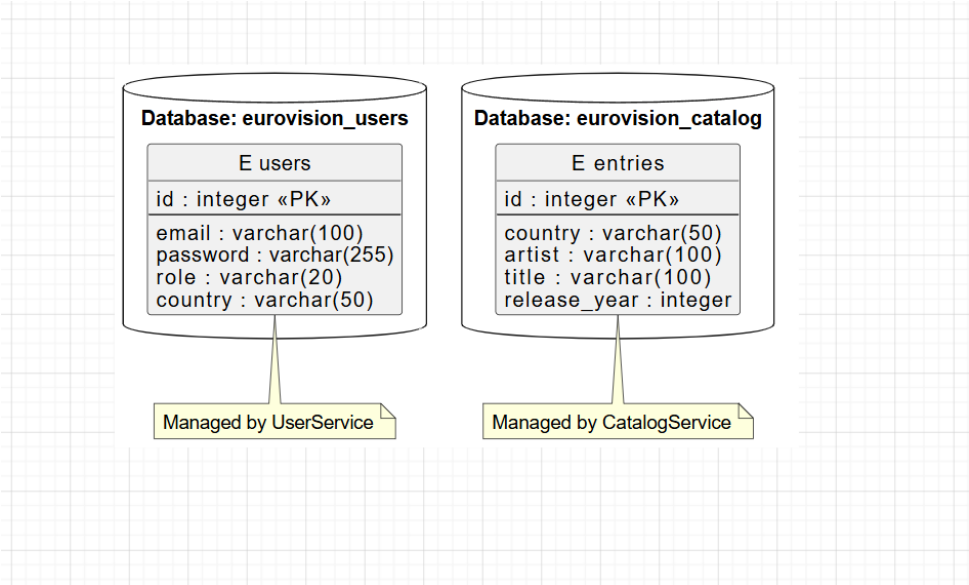


Figure 2: Entity-Relationship Diagram for Microservices

3.3 Client Application

The Client Application is responsible for presenting the user interface and handling user interactions. It uses the Fetch API for asynchronous REST communication with the back-end services and the WebSocket API for the live chat. It also manages state locally (using `sessionStorage` for user details) and handles internationalization.

3.4 Backend Services

The Backend is divided into two separate applications responsible for executing business logic, managing persistence via PostgreSQL, and enforcing authorization rules:

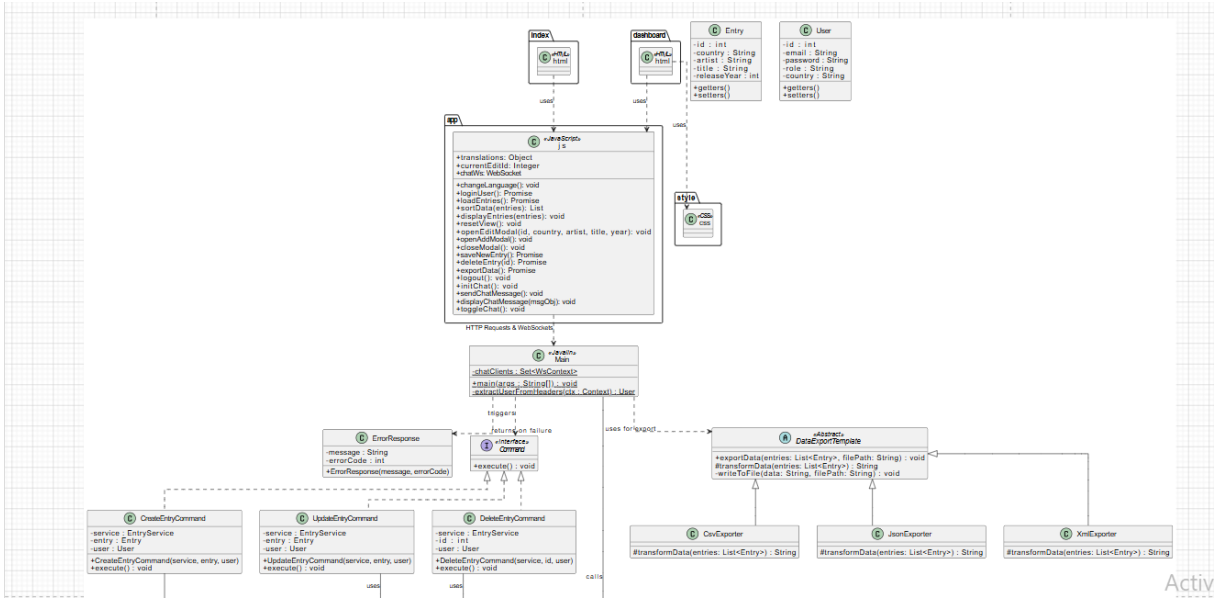
- **UserService:** Handles user authentication and role management.
- **CatalogService:** Handles catalog CRUD operations, data export (CSV, JSON, XML), and manages the WebSocket connections for the live chat feature.

3.5 Class diagrams

The class diagrams illustrate the internal organization of the Javalin server components, highlighting the structure of the entities, services, and the WebSockets handler for the chat feature.

3.6 Sequence diagrams

The sequence diagram illustrates the flow of the Real-Time Chat functionality: from a client sending a message payload via WebSocket, to the `CatalogService` receiving it, checking active sessions, and broadcasting the message iteratively to all connected `WsContext` clients.



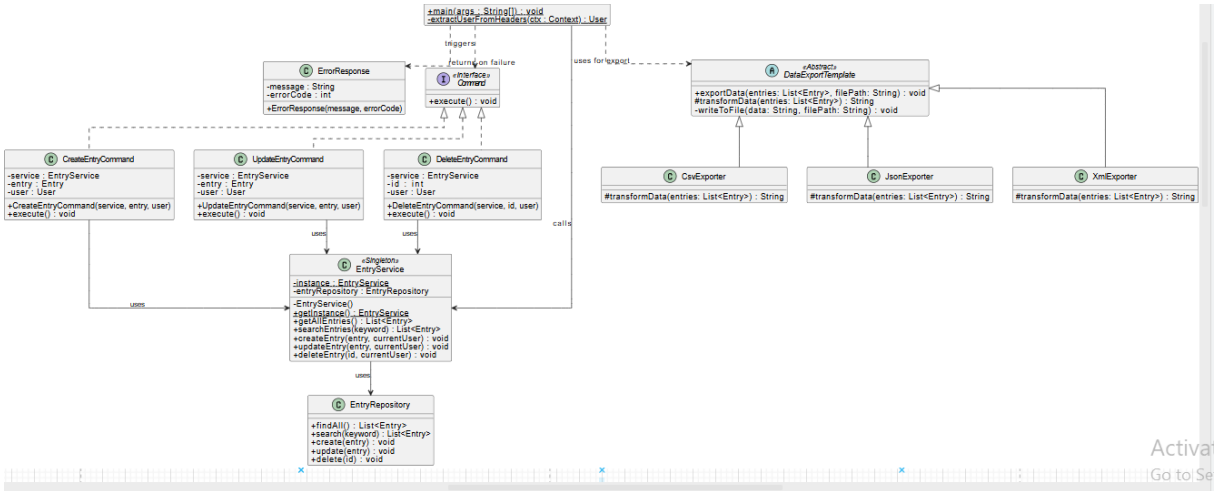


Figure 3: Class Diagram

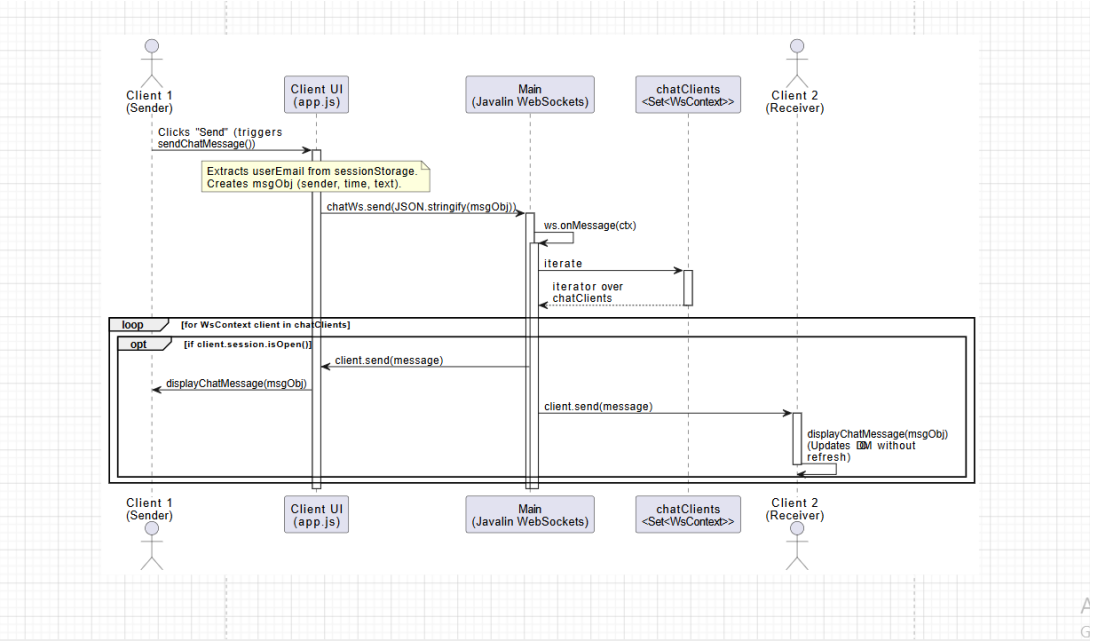


Figure 4: Sequence Diagram - Real-Time Chat Broadcasting

3.7 Design patterns

To ensure a maintainable and scalable backend architecture, specific design patterns were implemented within the Java services:

- **Command Pattern:** Applied to the CRUD operations (POST, PUT, DELETE) in the `CatalogService`. By encapsulating requests into standalone command objects (e.g., `CreateEntryCommand`, `DeleteEntryCommand`), the system decouples the Javalin route handlers from the core business logic, making the code easier to extend and test.
- **Template Method Pattern:** Utilized for the data export functionality. A standard interface defines the overall skeleton for exporting data, while concrete implementations (`CsvExporter`, `JsonExporter`, `XmlExporter`) provide the specific logic for formatting the Eurovision catalog into their respective file types.

4 Implementation phase - Application

4.1 Tools and technologies

- **Programming Language:** Java 21 (Backend Services), JavaScript (Client Application).
- **Web Framework:** Javalin - Chosen for its lightweight nature, simple CORS configuration, and built-in native support for WebSockets, which is crucial for the chat requirement.
- **Database:** PostgreSQL accessed via JDBC.
- **Communication Protocols:** HTTP/REST for standard CRUD operations, WebSockets for bidirectional real-time chat.

4.2 Application structure

The project is physically separated into three main modules to enforce the client-server architecture:

- `Eurovision_Client`: Contains the frontend application files (`index.html`, `dashboard.html`, `style.css`, `app.js`).
- `UserService`: The Java/Javalin backend module responsible for user authentication.
- `CatalogService`: The Java/Javalin backend module responsible for managing entries, exporting files (CSV, JSON, XML), and hosting the WebSocket chat.

4.3 User manual

The application features a clean, responsive graphical interface.

- **Authentication:** Users log in via the index page. Based on their role (Admin, HOD, Visitor), specific CRUD buttons are enabled or hidden dynamically.
- **Data Export:** Users can export the entire Eurovision catalog into CSV, JSON, or XML formats directly from the dashboard.
- **Internationalization:** A button in the header allows users to switch between English and Romanian instantly.
- **Live Chat:** A floating chat window resides in the bottom right corner. Clicking the header toggles its visibility. Users can send messages that appear instantly for all online users, displaying the sender's email and a timestamp.

5 Testing and validation

Testing ensures the application satisfies the analysis phase requirements:

- **API & Error Testing:** The REST endpoints were tested to ensure graceful error handling. Invalid requests return structured JSON errors with appropriate HTTP status codes, which the UI displays as user-friendly notifications without crashing.
- **Concurrency Testing:** The WebSocket chat was manually validated using multiple browser sessions to ensure stable real-time broadcasting, confirming that the server thread manages active connections properly without dropping other clients.
- **Unit Testing (JUnit):** Specific backend services were tested in isolation to verify data persistence and business logic.

References

[1] Oracle, *Java Documentation*. Available at: https://docs.oracle.com/en/java/

[2] Javalin, *A lightweight web framework for Java and Kotlin*. Available at: https://javalin.io/

[3] MDN Web Docs, *JavaScript*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[4] MDN Web Docs, *The WebSocket API (WebSockets)*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

[5] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Available at: https://www.postgresql.org/docs/